Insurance Analytics Project — Raw Data Generation

Objective

This notebook generates a synthetic insurance dataset to simulate a real-world environment for data analysis.

The dataset includes:

- Customers
- Policies
- Claims
- Payments

Why Synthetic Data?

Due to privacy and availability constraints in the insurance industry, synthetic data is used to:

- Mimic realistic business processes
- Enable end-to-end analytics workflows
- Introduce controlled data quality issues

Intentional Data Quality Issues

To reflect real-world datasets, the following issues are intentionally introduced:

- Missing values (e.g., income, claim_type)
- Duplicate records
- Inconsistent categorical values (e.g., "Auto", "auto", "AUTO")
- Mixed date formats
- Outliers in claim amounts
- Invalid foreign key relationships
- Incorrect data types
- Logical inconsistencies (e.g., claim before policy start)

These issues will be addressed in a separate data cleaning notebook.

Imports & Setup

In [1]:
```
#importing libraries
import pandas as pd
import numpy as np
from datetime import timedelta
import os

#Generating random seed
np.random.seed(42)
```

In [2]:
```
#genrating customers table
n_customers = 1000

customers = pd.DataFrame({
    "customer_id": range(1, n_customers + 1),
    "age": np.random.randint(18, 80, n_customers),
    "region": np.random.choice(["QC", "ON", "BC", "AB"], n_customers),
    "income": np.random.randint(30000, 120000, n_customers),
    "join_date": (
        pd.to_datetime("2018-01-01")
        + pd.to_timedelta(np.random.randint(0, 2000, n_customers), unit="D")
    )
})
```

In [3]:
```
#generating policies table
n_policies = 1500

#ensuring at least one policy per customer
base_policies = pd.DataFrame({
    "customer_id": customers["customer_id"],
    "policy_type": np.random.choice(["Auto", "Home", "Life"], n_customers),
    "premium_amount": np.random.randint(500, 3000, n_customers)
})

#generating additional policies
extra_policies = pd.DataFrame({
    "customer_id": np.random.choice(customers["customer_id"], n_policies - n_custom
    "policy_type": np.random.choice(["Auto", "Home", "Life"], n_policies - n_custom
    "premium_amount": np.random.randint(500, 3000, n_policies - n_customers)
})

#combining dataframes
policies = pd.concat([base_policies, extra_policies], ignore_index = True)
policies["policy_id"] = range(1, len(policies) + 1)

#assigning start dates based on customer join date
customer_join_map = customers.set_index("customer_id")["join_date"]

policies["start_date"] = (
    policies["customer_id"].map(customer_join_map)
    + pd.to_timedelta(np.random.randint(0, 30, len(policies)), unit = "D")
)

#assigning end dates
policies["end_date"] = policies["start_date"] + pd.to_timedelta(365, unit = "D")
```

In [4]:
```
#generating claims table
claims_list = []

for index, row in policies.iterrows():
    num_claims = np.random.poisson(0.5)

    for index in range(num_claims):
        claim_amount = np.random.gamma(2, 1000)

        claims_list.append({
            "policy_id": row["policy_id"],
            "claim_amount": round(claim_amount, 2),
            "claim_date": (
                row["start_date"]
                + pd.to_timedelta(np.random.randint(0, 365), unit = "D")
            ),
            "claim_type": np.random.choice(["Collision", "Theft", "Fire"])
        })

claims = pd.DataFrame(claims_list)
claims["claim_id"] = range(1, len(claims) + 1)
```

In [5]:
```
#generating payments table
payments_list = []

for index, row in policies.iterrows():
    #monthly payments for 12 months
    for month in range(12):
        payment_date = (
            row["start_date"]
            + pd.DateOffset(months = month)
        )

        payments_list.append({
            "policy_id": row["policy_id"],
            "payment_amount": row["premium_amount"] / 12,
            "payment_date": payment_date
        })

payments = pd.DataFrame(payments_list)
payments["payment_id"] = range(1, len(payments) + 1)
```

Clean Reference Data

Before introducing data quality issues, a clean version of each table will be preserved.

This allows:

- Validation of cleaning steps
- Before vs after comparisons
- Easier debugging

In [6]:
```
customers_clean = customers.copy()
policies_clean = policies.copy()
claims_clean = claims.copy()
payments_clean = payments.copy()
```

Data Issues

The following issues are intentionally introduced to simulate real-world datasets.

In [7]:
```
#missing values
customers.loc[customers.sample(frac = 0.1).index, "income"] = None
claims.loc[claims.sample(frac = 0.1).index, "claim_type"] = None

#duplicates
customers = pd.concat([customers, customers.sample(50)], ignore_index = True)

#inconsistent categories
inconsistent_policy_idx = policies.sample(frac = 0.2).index
policies.loc[inconsistent_policy_idx, "policy_type"] = np.random.choice(["auto", "A

#mixed date formats
customers["join_date"] = customers["join_date"].astype(str)
inconsistent_date_idx = customers.sample(frac = 0.3).index
customers.loc[inconsistent_date_idx, "join_date"] = pd.to_datetime(customers.loc[in

#outliers
claims.loc[claims.sample(10).index, "claim_amount"] = 100000

#broken foreign keys
claims.loc[claims.sample(5).index, "policy_id"] = 999999

#wrong data types
customers["income"] = customers["income"].astype(str)

#logical errors
#policy before customer join (breaking 10%)
invalid_policy_date_idx = policies.sample(frac = 0.1).index
policies.loc[invalid_policy_date_idx, "start_date"] = (
    policies.loc[invalid_policy_date_idx, "start_date"]
    -pd.to_timedelta(500, unit="D")
)

#negative claim amounts
claims.loc[claims.sample(5).index, "claim_amount"] = -500
```

Export Raw Data

The generated dataset (including intentionally introduced data quality issues) is exported to the `data/raw/` directory.

This represents the **raw input layer** of the data pipeline and will be used in the data cleaning and transformation phase.

Separating raw data ensures:

- Reproducibility
- Clear data lineage
- Safe handling of original data

In [8]:
```
#creating directory structure if it doesn't exist
os.makedirs("data/raw", exist_ok = True)
os.makedirs("data/reference", exist_ok=True)

#exporting original clean data for reference
customers_clean.to_csv("data/reference/customers_clean.csv", index=False)
policies_clean.to_csv("data/reference/policies_clean.csv", index=False)
claims_clean.to_csv("data/reference/claims_clean.csv", index=False)
payments_clean.to_csv("data/reference/payments_clean.csv", index=False)

#exporting raw datasets
customers.to_csv("data/raw/customers.csv", index = False)
policies.to_csv("data/raw/policies.csv", index = False)
claims.to_csv("data/raw/claims.csv", index = False)
payments.to_csv("data/raw/payments.csv", index = False)

print("Raw datasets exported successfully")
```

```
Raw datasets exported successfully
```

Summary

A synthetic insurance dataset was generated with realistic relationships and intentionally introduced data quality issues.

The dataset has been exported to `data/raw/` and will be cleaned and transformed in the next notebook.

In [ ]: